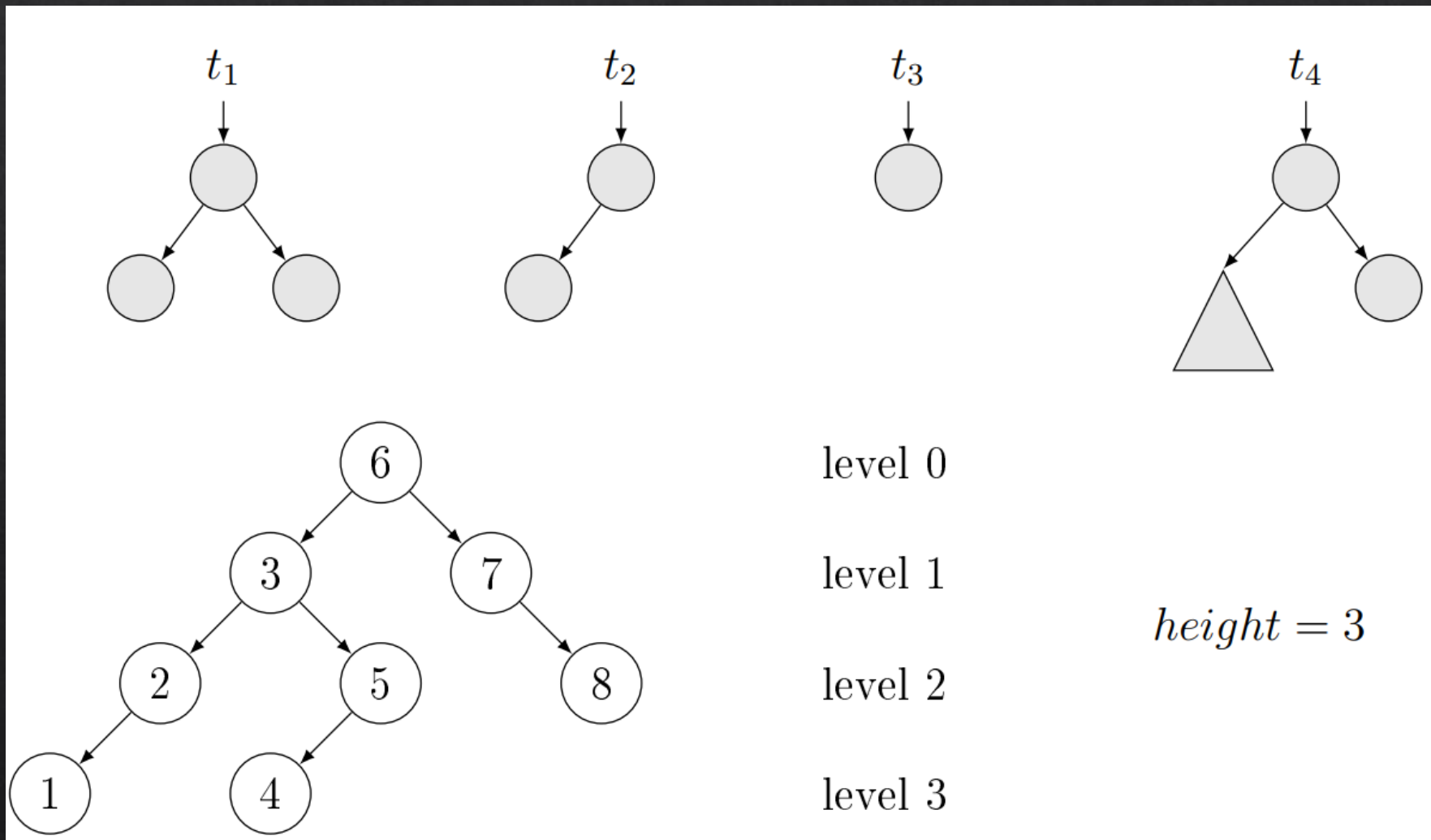


Algorithms and data structures I.

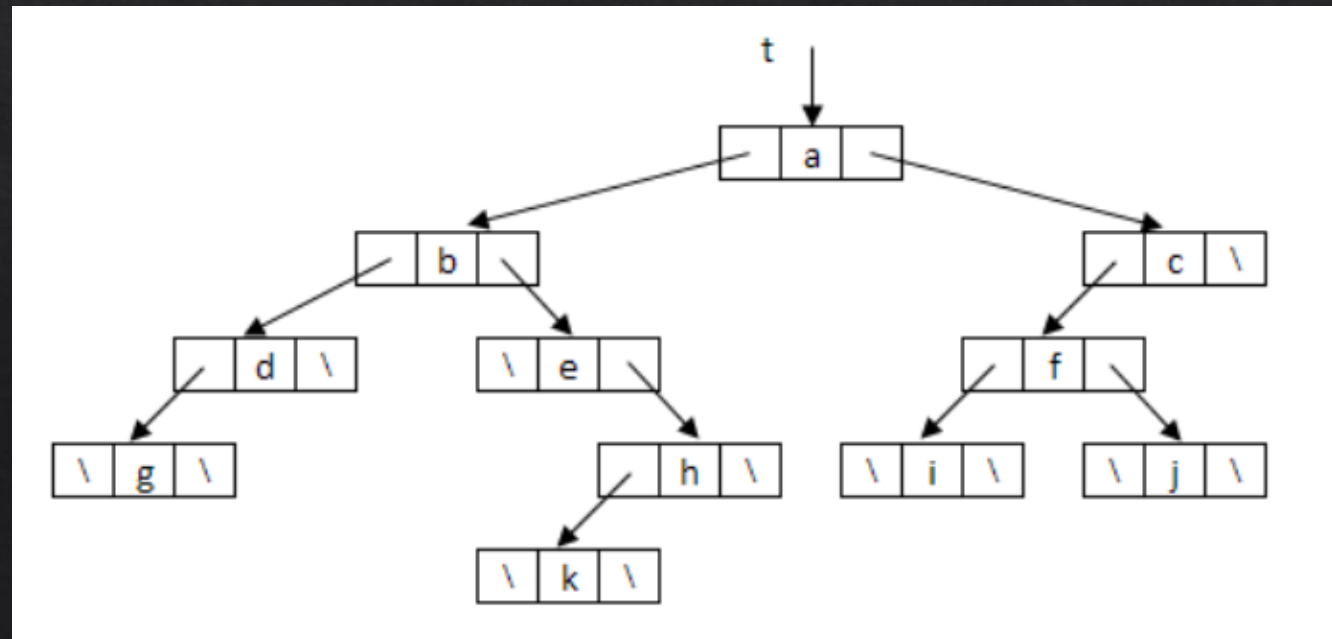
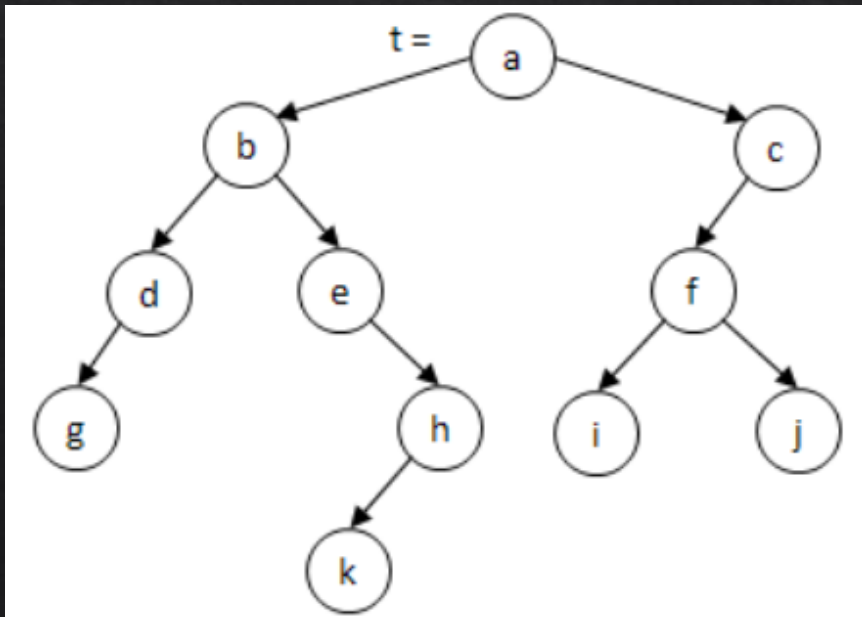
Trees

Tree - data structure



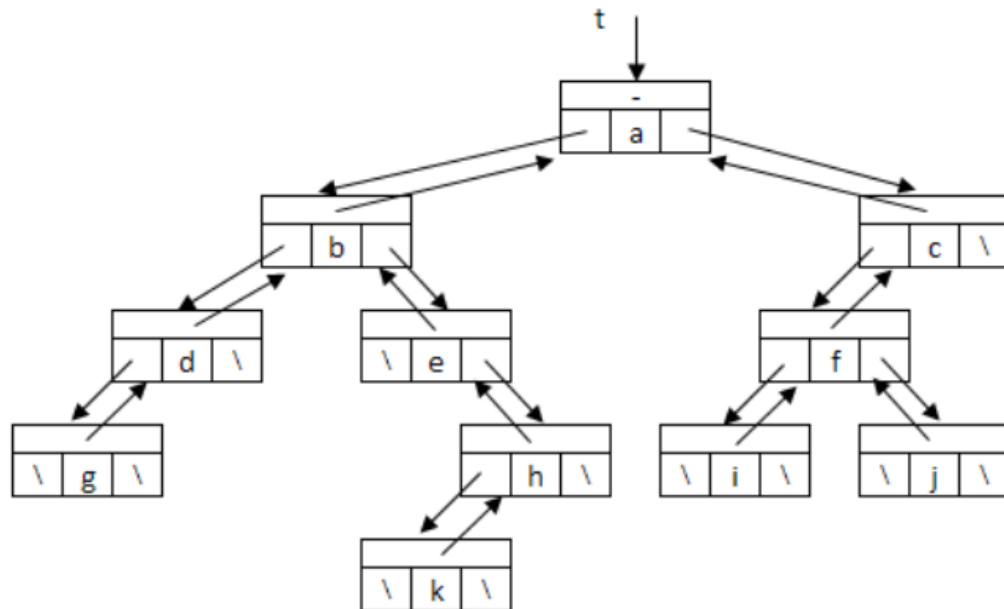
Binary tree - (a tree with max 2 child)

Node
+ $key : \mathcal{T}$
+ $left, right : \text{Node}^*$
+ $\text{Node}() \{ left := right := \emptyset \}$ // Single element tree
+ $\text{Node}(x : \mathcal{T}) \{ left := right := \emptyset ; key := x \}$



Binary tree - with parent pointer

Node3
+ $key : \mathcal{T} // \mathcal{T}$
+ $left, right, parent : \text{Node3}^*$
+ $\text{Node3}(p:\text{Node3}^*) \{ left := right := \emptyset ; parent := p \}$
+ $\text{Node3}(x : \mathcal{T}, p:\text{Node3}^*) \{ left := right := \emptyset ; parent := p ; key := x \}$



Implementation	Theory	Meaning
$t \rightarrow \text{left}$	$\text{left}(t)$	Left subtree of 't'
$t \rightarrow \text{right}$	$\text{right}(t)$	Right subtree of 't'
$t \rightarrow \text{parent}$	$\text{parent}(t)$	Parent of 't'
$t \rightarrow \text{key}$	$\text{key}(t)$	Key value of 't'
$t = 0, \text{NIL}, \text{NULL}$	$t = \Omega$	Empty tree

Traversing binary trees

◇ Traversing practically means enumerating the elements in the tree.

◇ **Recursive traversals**¹:

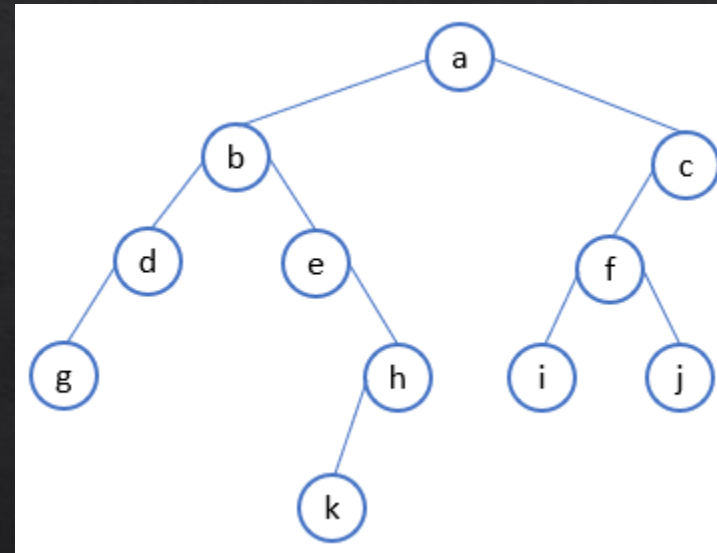
◇ Preorder

◇ Inorder

◇ Postorder

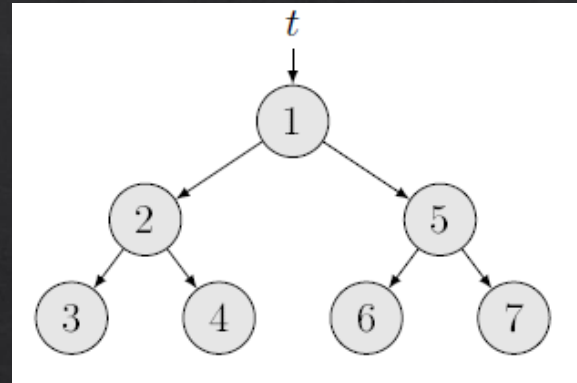
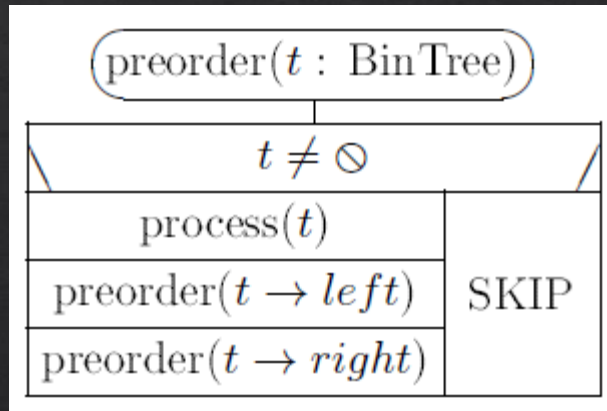
◇ Level order:

In case of general graphs, this is equivalent of BFS (see next semester).

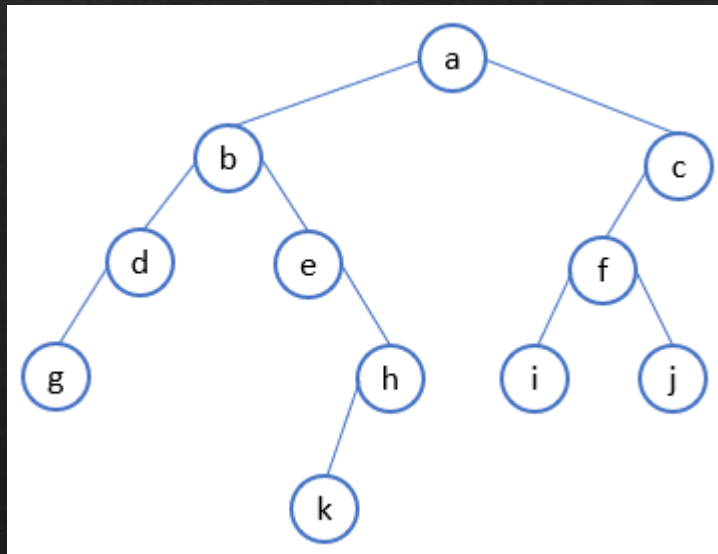


¹ Implementable iteratively with a stack

Preorder



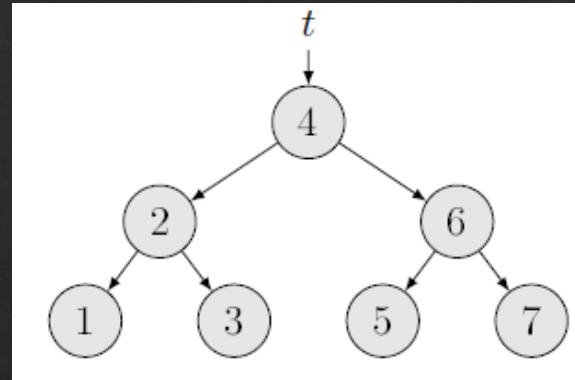
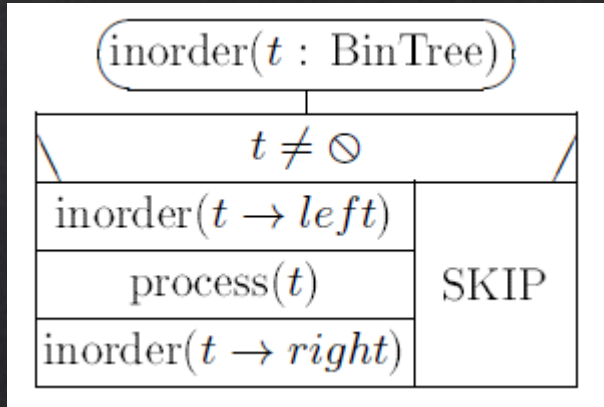
Order of
accessing
elements



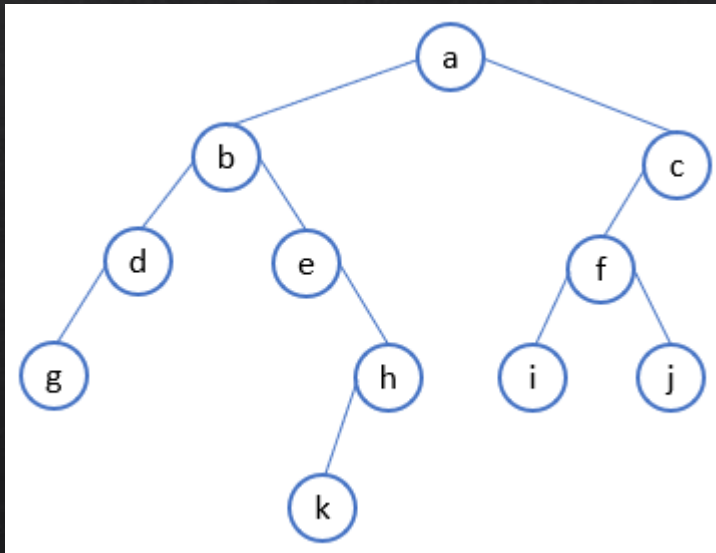
Preorder output:

a b d g e h k c f i j

Inorder



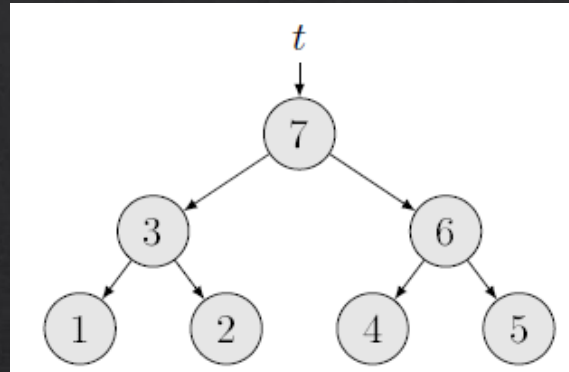
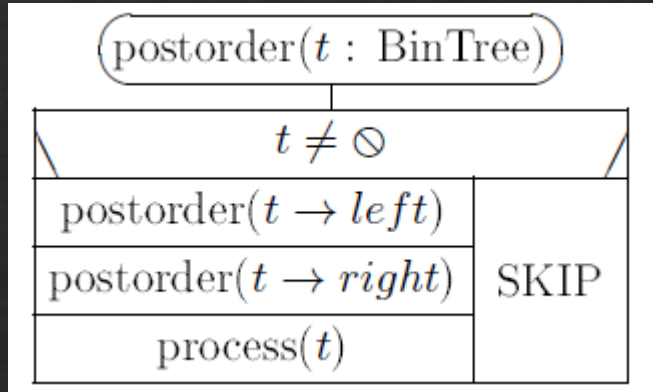
Order of
accessing
elements



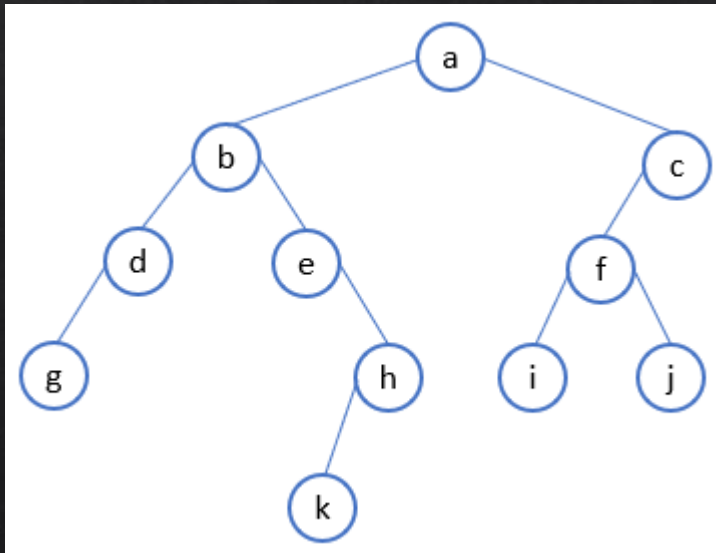
Inorder output:

g d b e k h a i f j c

Postorder



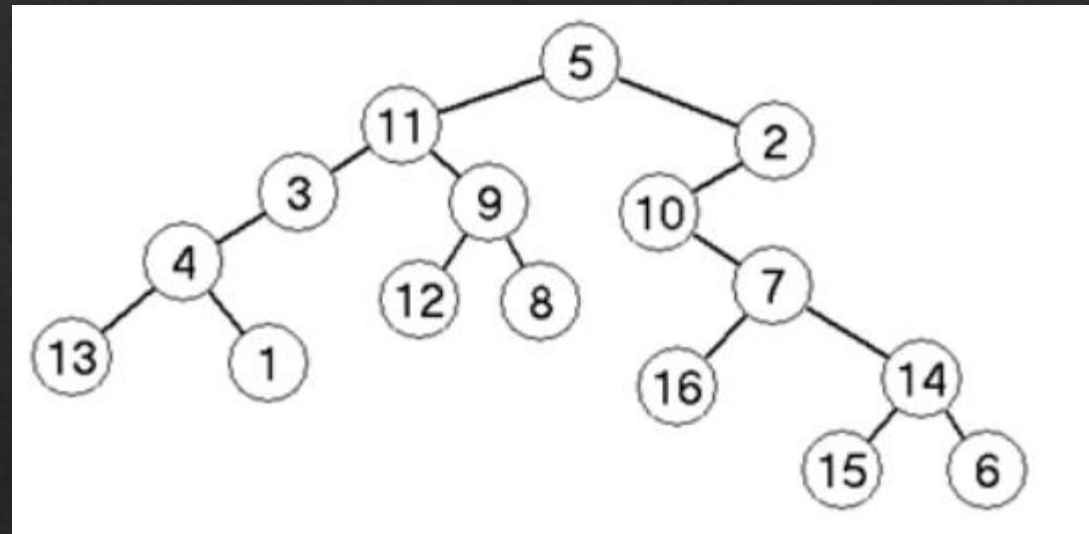
Order of
accessing
elements.



Postorder output:

g d k h e b i j f c a

Exercise: Provide the outputs of all three traversals on the tree below



Preorder: 5, 11, 3, 4, 13, 1, 9, 12, 8, 2, 10, 7, 16, 14, 15, 6

Inorder: 13, 4, 1, 3, 11, 12, 9, 8, 5, 10, 16, 7, 15, 14, 6, 2

Postorder: 13, 1, 4, 3, 12, 8, 9, 11, 16, 15, 6, 14, 7, 10, 2, 5

Exercise: Reconstructing the tree from the output

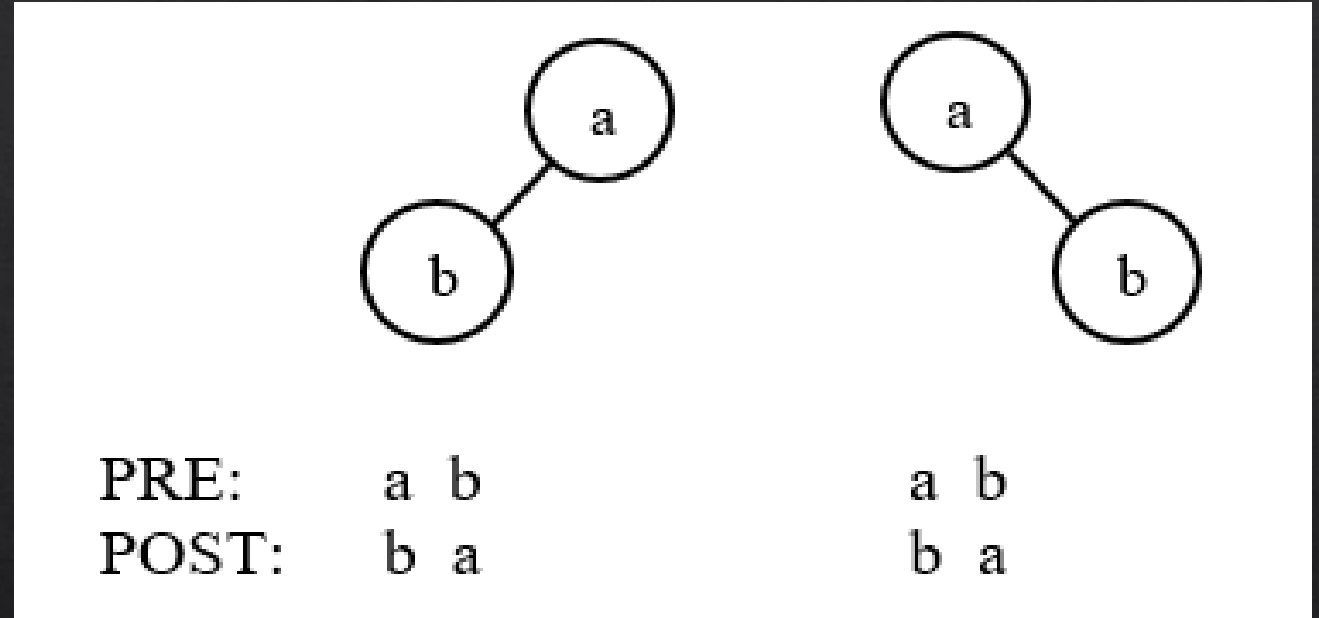
POSTORDER: 17, 4, 7, 9, 45, 91, 2, 8, 53, 10, 66, 72, 34, 5, 3
INORDER: 17, 8, 4, 7, 91, 45, 9, 2, 3, 5, 34, 53, 66, 10, 72

- ◊ Which is the root?
- ◊ Postorder processes it last → 3 is the root
- ◊ What information can we extract from the inorder traversal?
- ◊ It always processes all the nodes of the left subtree before the root → Every element to the left of 3 is in the left subtree of 3 and every element to the right of 3 is in the right subtree of 3.
- ◊ In the blue box we find the postorder traversal of the left subtree, in the purple box the postorder traversal of the right subtree.
- ◊ In the orange box we find the inorder traversal of the left subtree, in the green box the inorder of the right subtree.
- ◊ Continue recursively.

POSTORDER: 17, 4, 7, 9, 45, 91, 2, 8, 53, 10, 66, 72, 34, 5, 3
INORDER: 17, 8, 4, 7, 91, 45, 9, 2, 3, 5, 34, 53, 66, 10, 72

Can we reconstruct the tree from any of the two traversals?

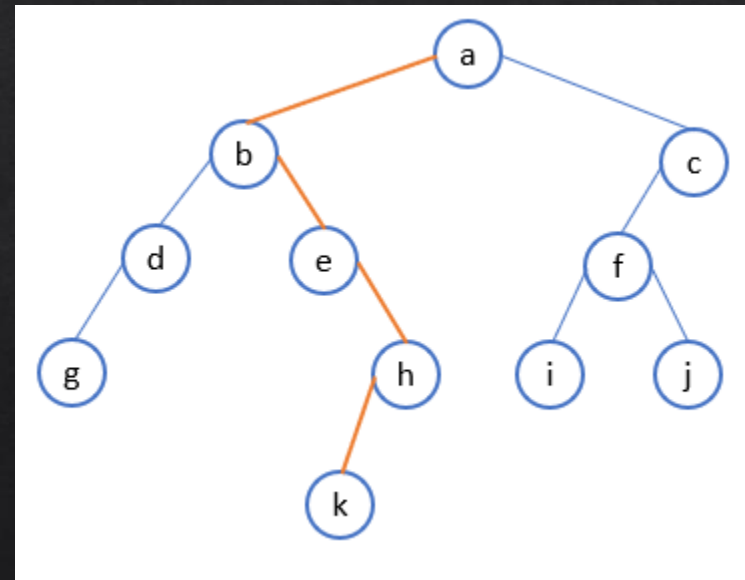
- ◇ No!
- ◇ Which two does not work?
- ◇ Answer: pre-, and postorder!
- ◇ Counterexample on the right image.



Height of the binary tree

Height of the binary tree

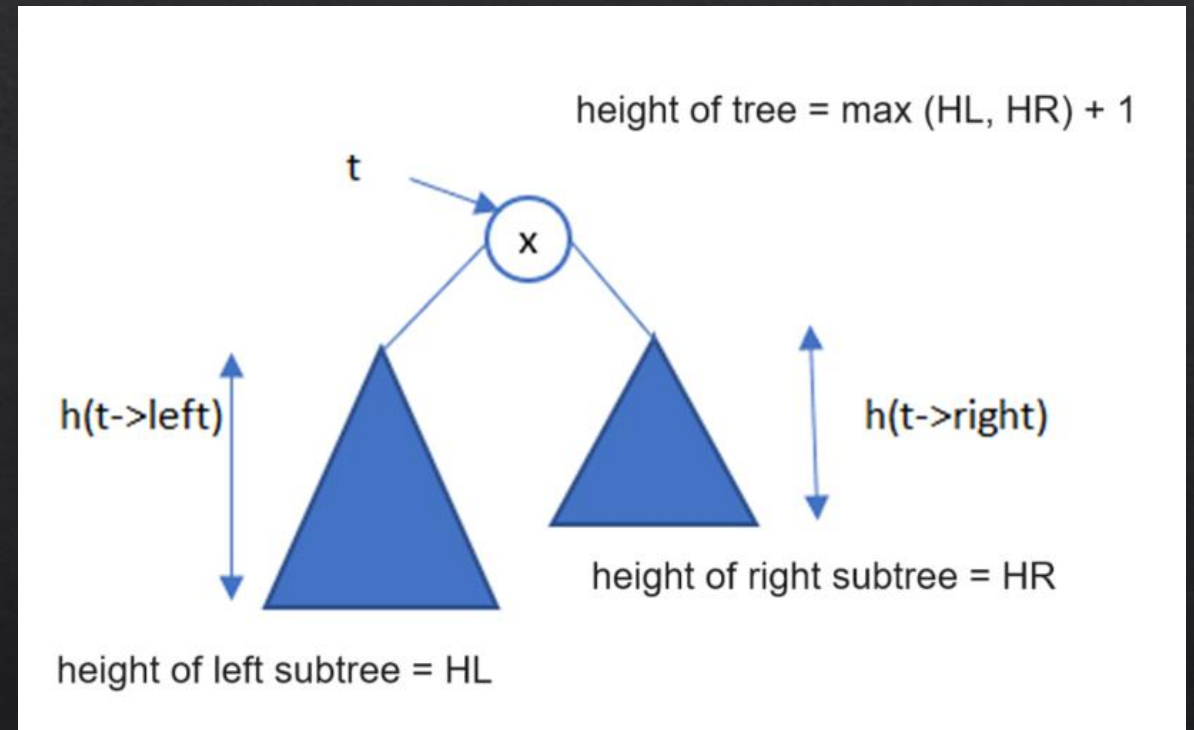
- ◆ Definition: the length of one of the longest 'arms' or 'branches' of the binary tree is called the *height* of the binary tree.
- ◆ Height of the empty tree is: -1.
- ◆ Height of a tree with a single node: 0
- ◆ Height of a tree with two nodes: 1
- ◆ Height of the tree in the example to the right: 4



Task: Calculate the height of a tree

Task: Calculate the height of a tree

- Exercise: design an algorithm which calculates the height of the binary tree.
- Idea: recursion!
- The height of a tree is 1 plus the maximum of the heights of its left and right subtrees.
- $\text{Height} = \max(\text{HL}, \text{HR}) + 1$
- This is true for all subtrees recursively.
- End of the recursion: empty tree, where the height is -1.



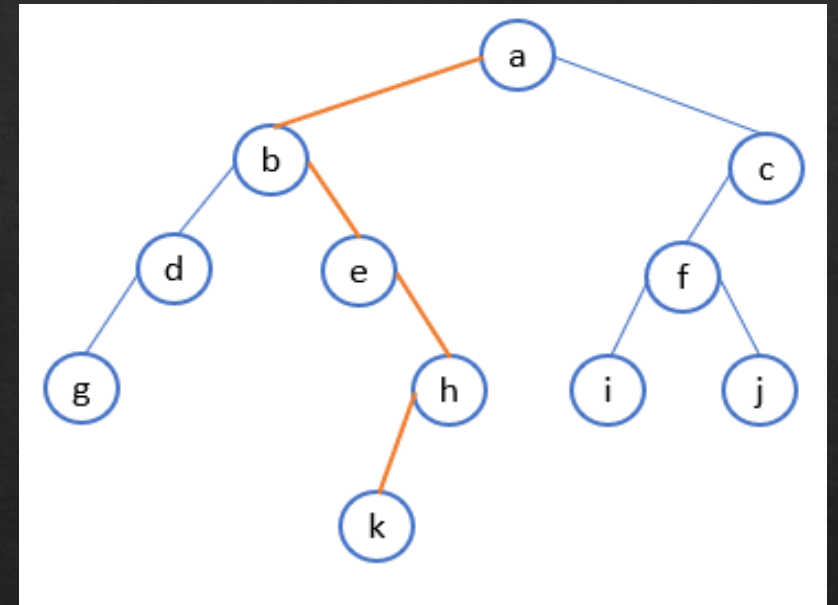
Algorithm: Calculate the height of a tree

The header of the structogram can be:

$h(t: \text{Node}^*): \mathbb{Z}$ or $h(t: \text{Bintree}): \mathbb{Z}$

$h(t: \text{Bintree}): \mathbb{Z}$

$t \neq 0$	
$HL := h(t \rightarrow \text{left})$	return -1
$HR := h(t \rightarrow \text{right})$	
return $\max(HL, HR) + 1$	



Task: Number of leaves in a binary tree

Number of leaves in a binary tree: first try

- ◆ Beware of initial ideas:

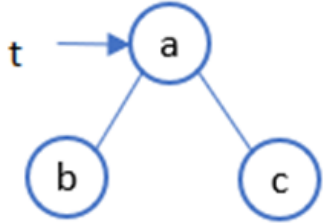
- ◆ Number of leaves in the left subtree: LL, in the right subtree: RL
- ◆ Total number of leaves: LL+LR
- ◆ Number of leaves in the empty tree: 0

LeafNo(t:Bintree):N	
t ≠ 0	
LL:=LeafNo(t->left)	return 0
LR:=LeafNo(t->right)	
return LL + LR	

Cases:

Empty tree

Tree with 1 node: 

Tree with 3 nodes: 

- ◆ **NOT GOOD**: this algorithm returns 0 all the time.

Number of leaves in a binary tree: second try

LeafNo (t:Bintree):N

t ≠ 0	
LL:=LeafNo (t->left)	return 0
LR:=LeafNo (t->right)	
return LL+LR+1	

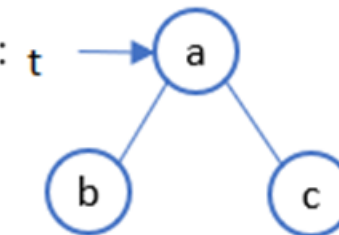
Cases:

Empty tree:

Tree with 1 node:



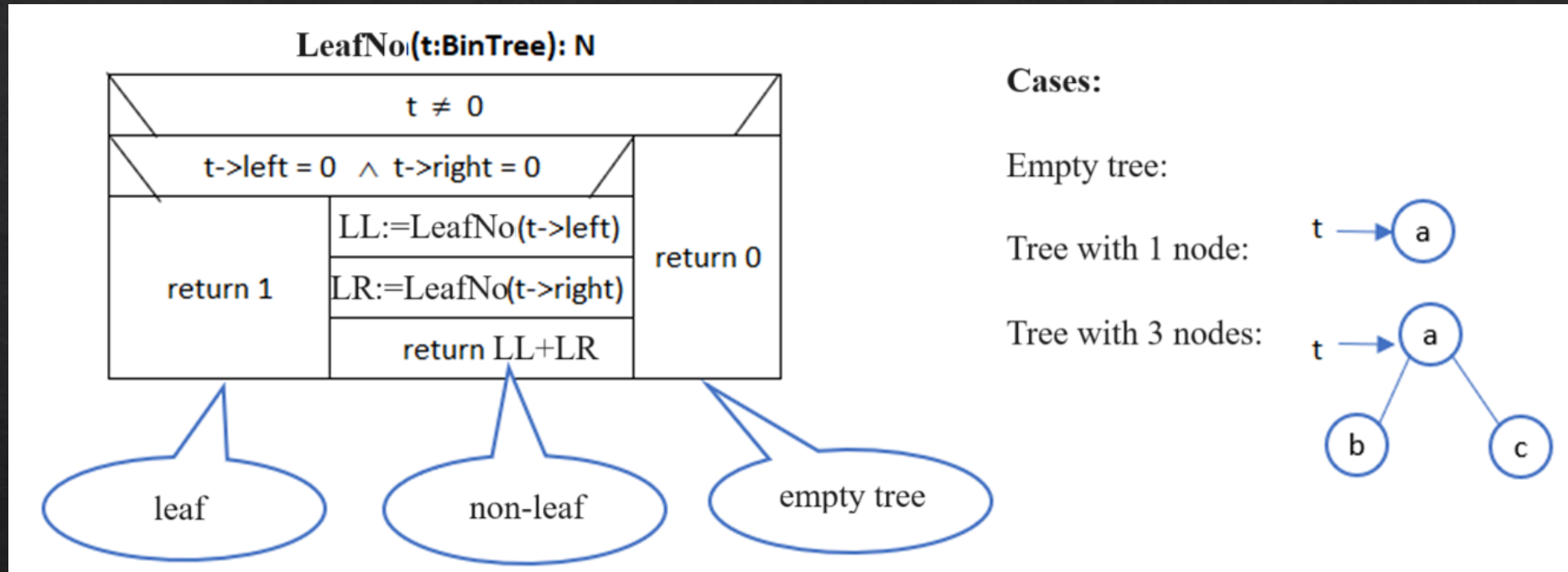
Tree with 3 nodes:



Still **NOT GOOD**.

At least its not all zeros, but it returns something else!
It calculates the total number of elements in the tree.

Number of leaves in a binary tree: final solution

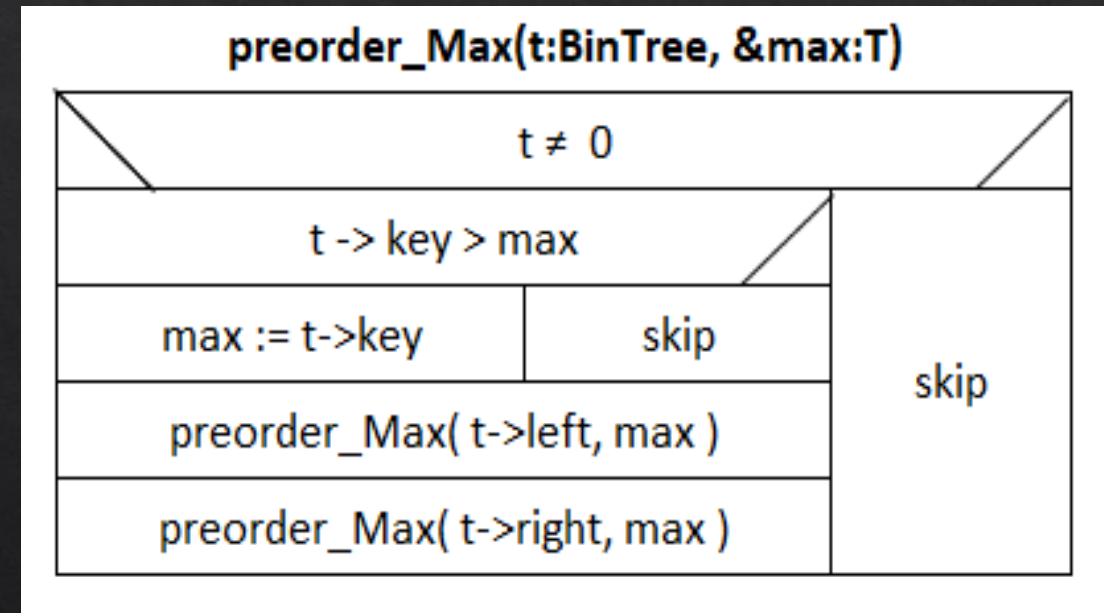
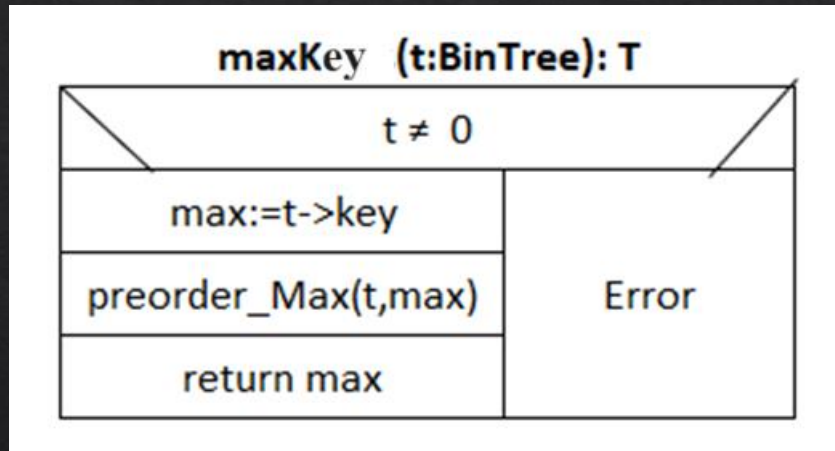


Conclusion: Different 'types' of nodes in the tree might require different treatment.

Task: Finding maximum key in a binary tree

Finding maximum key in a binary tree

- ❖ **First solution:** Passing max argument as a reference so it can be updated during traversal!
- ❖ Empty trees must be treated separately. The main algorithm starts with the key value of the root as maximum value.



This can be generalized to nearly any basic programming task.

Finding maximum key in a binary tree

Second solution: Instead of reference, use the approach presented at the leaf counting problem, and use return value.

We need a starting algorithm here as well to handle empty trees.

maxKey (t:BinTree):T

t=0	
Error	return(maxFind(t))

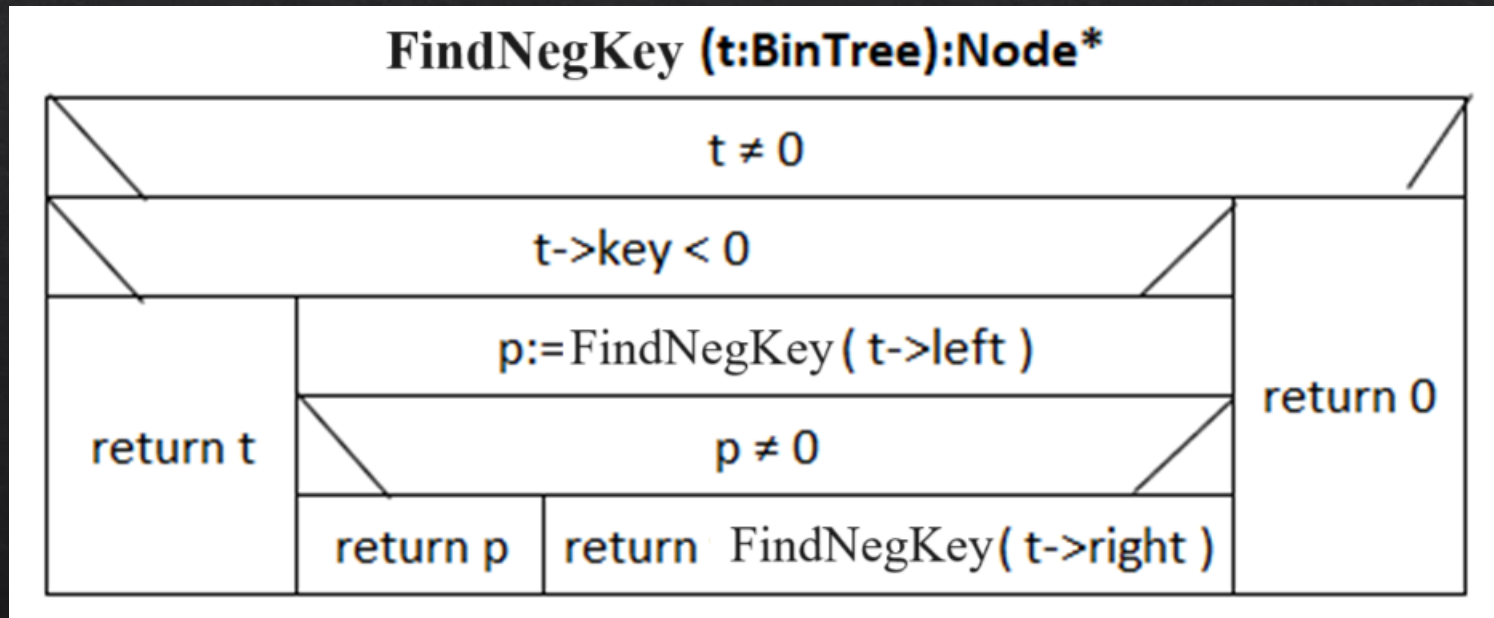
max Find (t:BinTree):T

t->left =0 és t->right=0			
return t->key	t->left≠0 ∧ t->right≠0	t->left≠0 ∧ t->right=0	t->left=0 ∧ t->right≠0
	max1:=max Find (t->left)	max1:=max Find (t->left)	max1:=max Find (t->right)
	max2:=max Find (t->right)		
	max1:=max(max1,max2)		
	return max(max1,t->key)		

Is there a node with negative key value in the bintree?

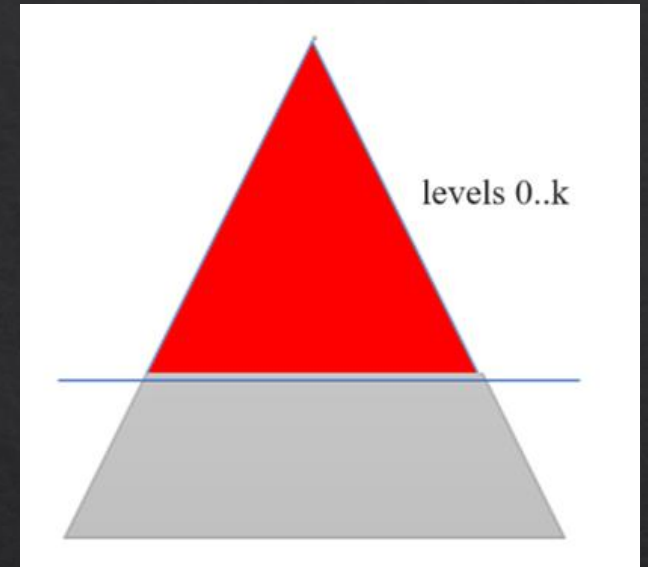
If yes, then return its address, if no, return 0.

- ◆ **Search.** At any point during the traversal if we find a negative value we can stop the traversal, and return it's address. If the traversal ended then we return nullptr, this way we can omit the boolean aspect of these type of algorithms.



Task: Count the number of nodes
having a single child

Count the number of nodes
on levels 0..k having a single child

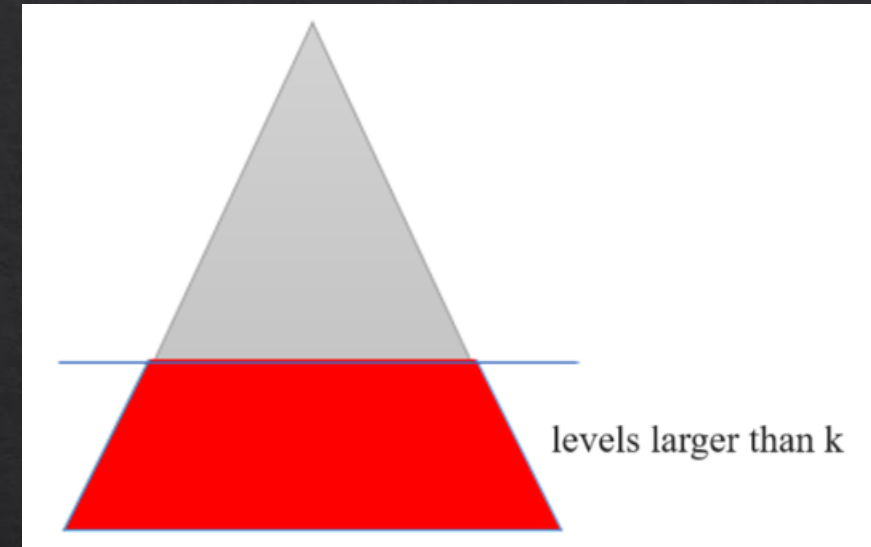


SingleChild (t:Bintree, k: N) : N

t = 0 OR k < 0		
return 0	c1:= SingleChild(t->left,k-1)	
	c2:=SingleChild(t->right,k-1)	
	(t->left≠0 & t->right=0) OR (t->left=0 & t->right≠0)	
	return c1+c2+1	return c1+c2

Note: k is not passed as
reference!

Count the number of nodes
which are on the levels $> k$
having a single child

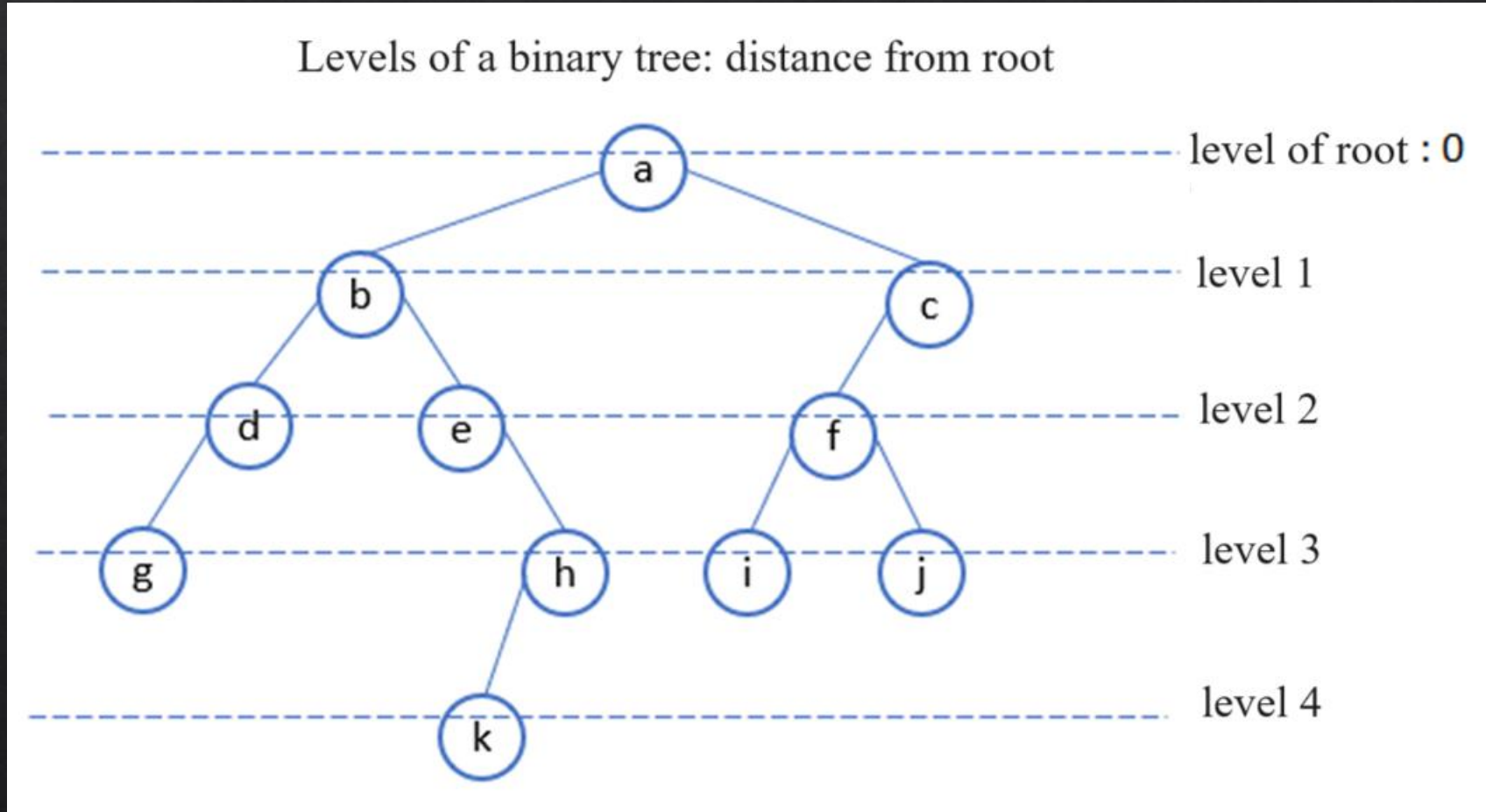


SingleChild (t: Bintree, k: N) : N

t = 0		
return 0	c1:=SingleChild(t->left,k-1)	
	c2:=SingleChild(t->right,k-1)	
	k < 0 & ((t->left≠0 & t->right=0) OR (t->left=0 & t->right≠0))	
	return c1+c2+1	return c1+c2

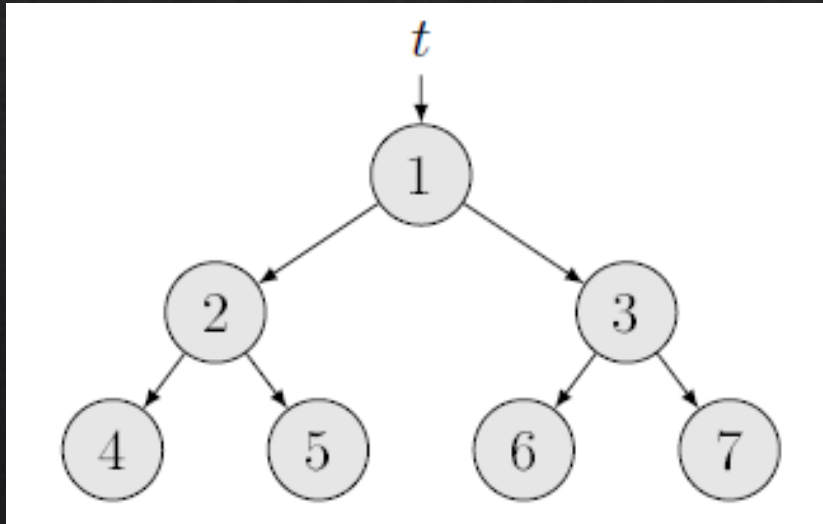
Levels of a binary tree

Levels of a binary tree



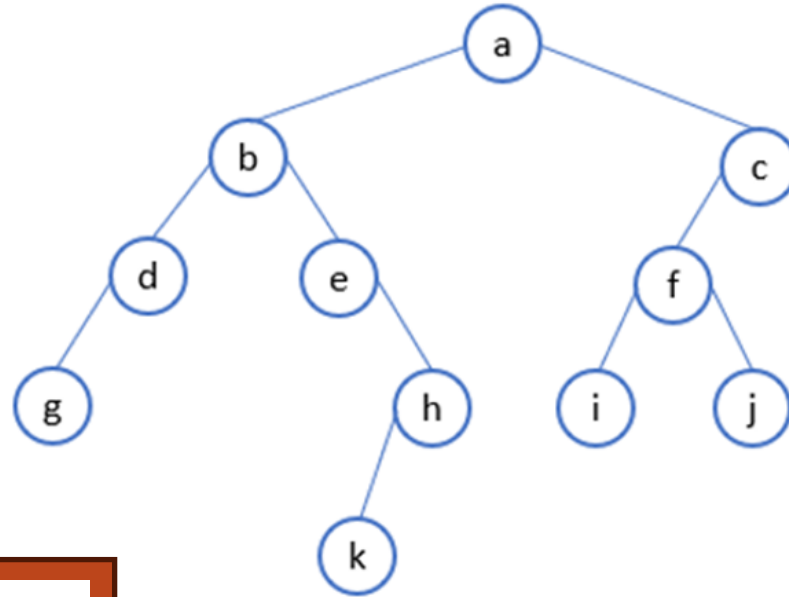
A 4th traversal: level order traversal

- ◆ This is the algorithm in the lecture notes.
- ◆ Order of accessed elements:



levelOrder(t : BinTree)

$t \neq \emptyset$	
$Q : \text{Queue} ; Q.\text{add}(t)$	SKIP
$\neg Q.\text{isEmpty}()$	
$s := Q.\text{rem}()$	
$\text{process}(s)$	
$s \rightarrow \text{left} \neq \emptyset$	
$Q.\text{add}(s \rightarrow \text{left})$	
$s \rightarrow \text{right} \neq \emptyset$	
$Q.\text{add}(s \rightarrow \text{right})$	



Initialization:

Node processed:

a

b

c

d

e

f

g

h

i,j,k

contents of queue Q:

a

b c

c d e

d e f

e f g

f g h

g h i j

h i j

i j k

levelOrder(t : BinTree)

$t \neq \emptyset$

Q : Queue ; $Q.add(t)$

$\neg Q.isEmpty()$

$s := Q.rem()$

process(s)

$s \rightarrow left \neq \emptyset$

$Q.add(s \rightarrow left)$ SKIP

$s \rightarrow right \neq \emptyset$

$Q.add(s \rightarrow right)$ SKIP

SKIP

Task:

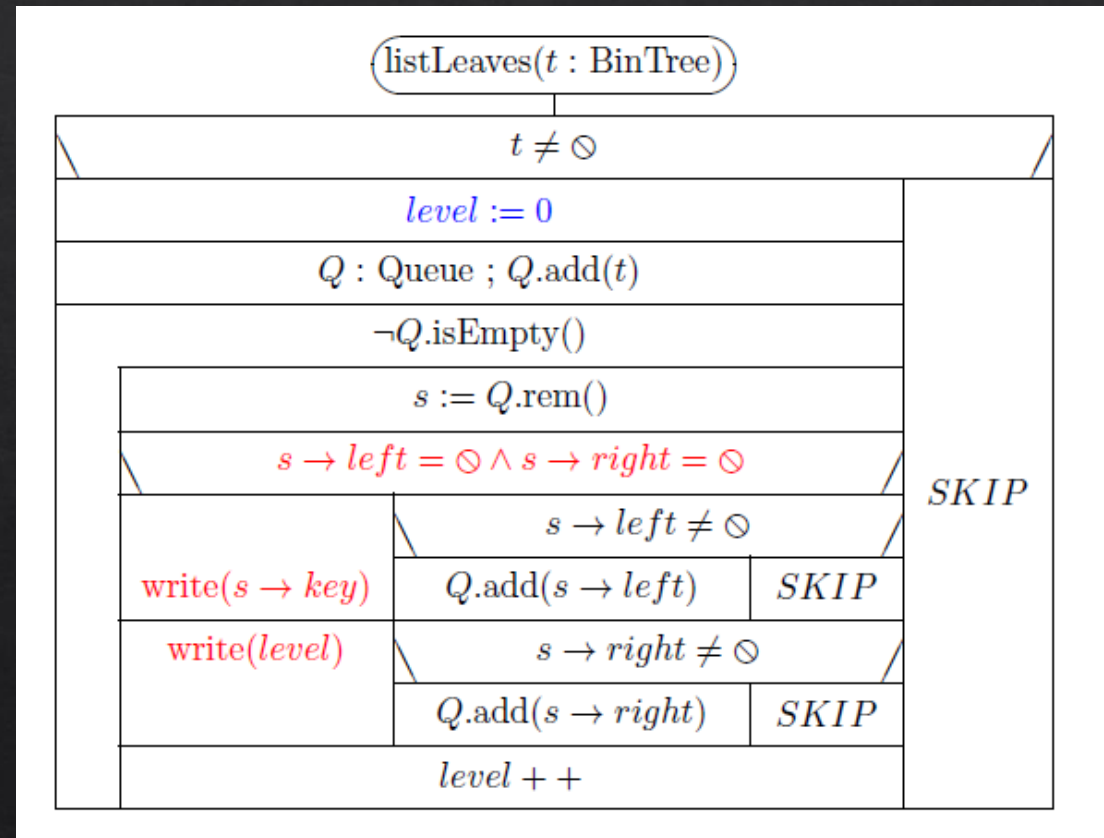
Print the key values and level numbers of all leaves

Print the key values and level numbers of all leaves

- ◆ This algorithm tries to solve this problem: printing the key values and level numbers of all leaves.
- ◆ Try to demonstrate it on a tree with at least 2 levels and 3 nodes.

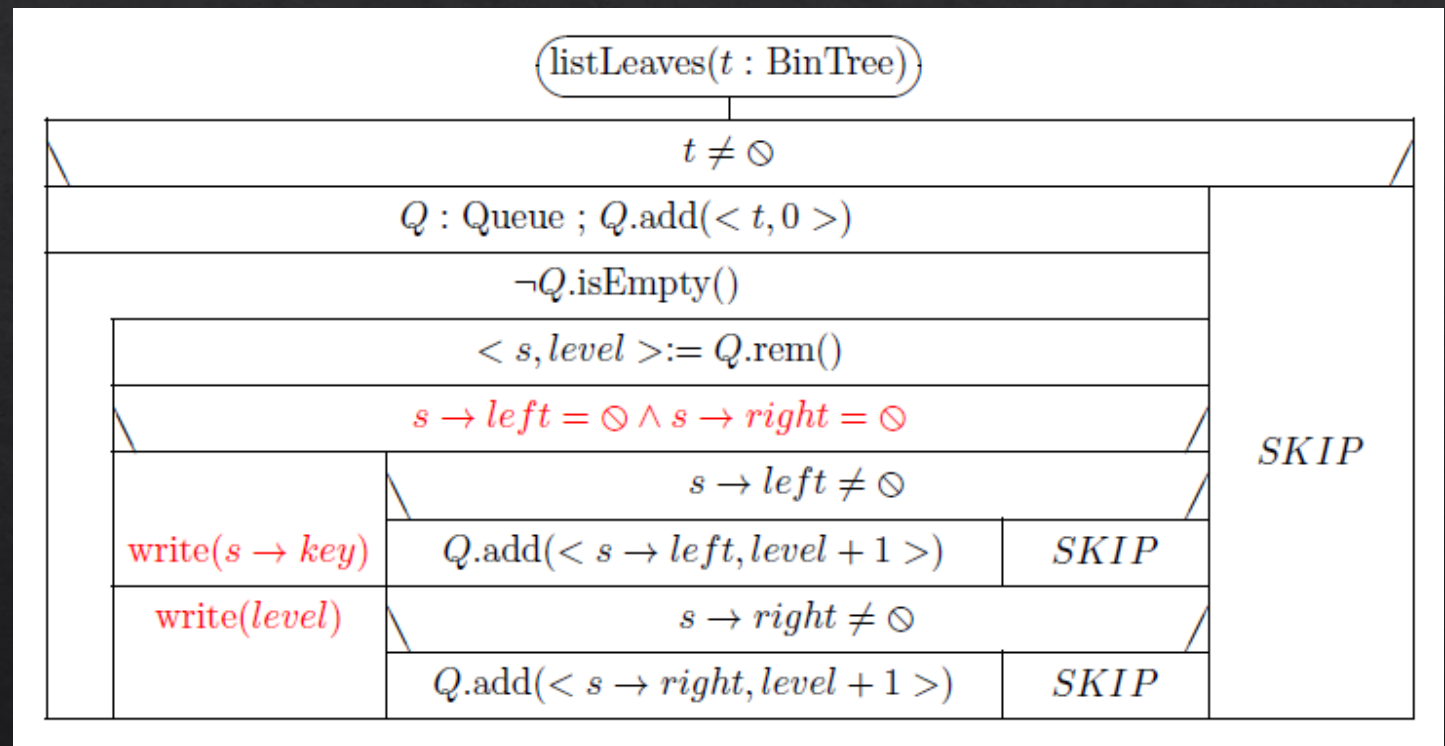
What's wrong?

- ◆ Problem: instead of level numbers this prints the order of appearance.



Second attempt to print the key values and level numbers of all leaves:

- ◆ This time we add $\langle \text{Node}^*, N \rangle$ tuples to the queue, where N is the level number of the given node.
- ◆ This really solves this problem.
- ◆ Do we really need these tuples?
- ◆ No, there are two approaches to solve the problem without tuples.



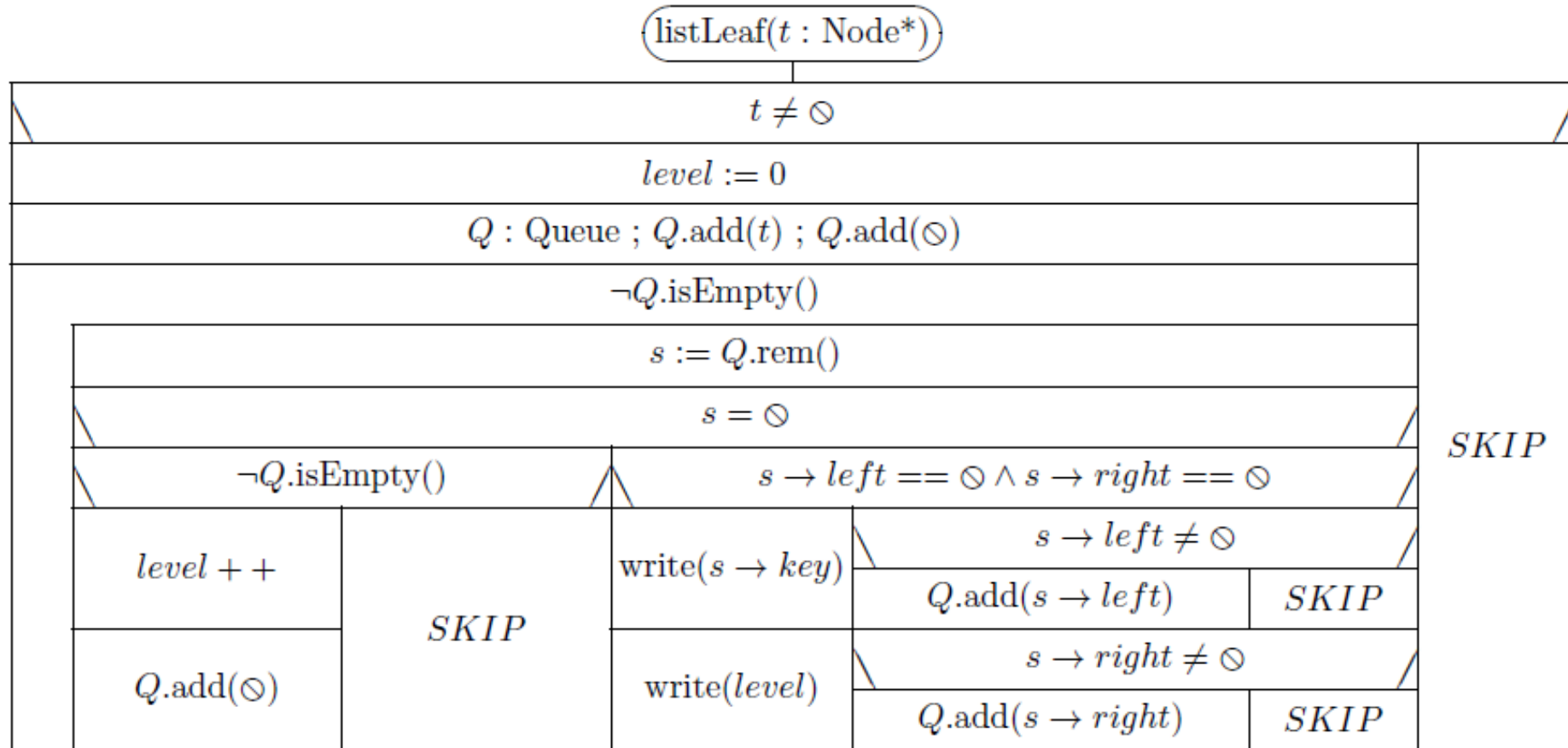
◆ First approach:

- ◆ End marks: in the queue we keep track of where a level ends and where the next one starts by using an end mark (here a NULL pointer). At the end of every level we insert an end mark (NULL) into the queue.
- ◆ We start by adding NULL after the root, and from then on: whenever we reach a NULL in the queue, we remove it, then add it again and increase the level number.

◆ Second approach:

- ◆ We keep track of the elements on every level by:
 - ◆ a) knowing the number of elements on the current level (at first this is 1, from then on it is passed on from the previous iteration),
 - ◆ b) keeping track of how many elements have been processed so far,
 - ◆ c) counting how many elements are added to the queue on this level – this will be the number of elements on the next level.

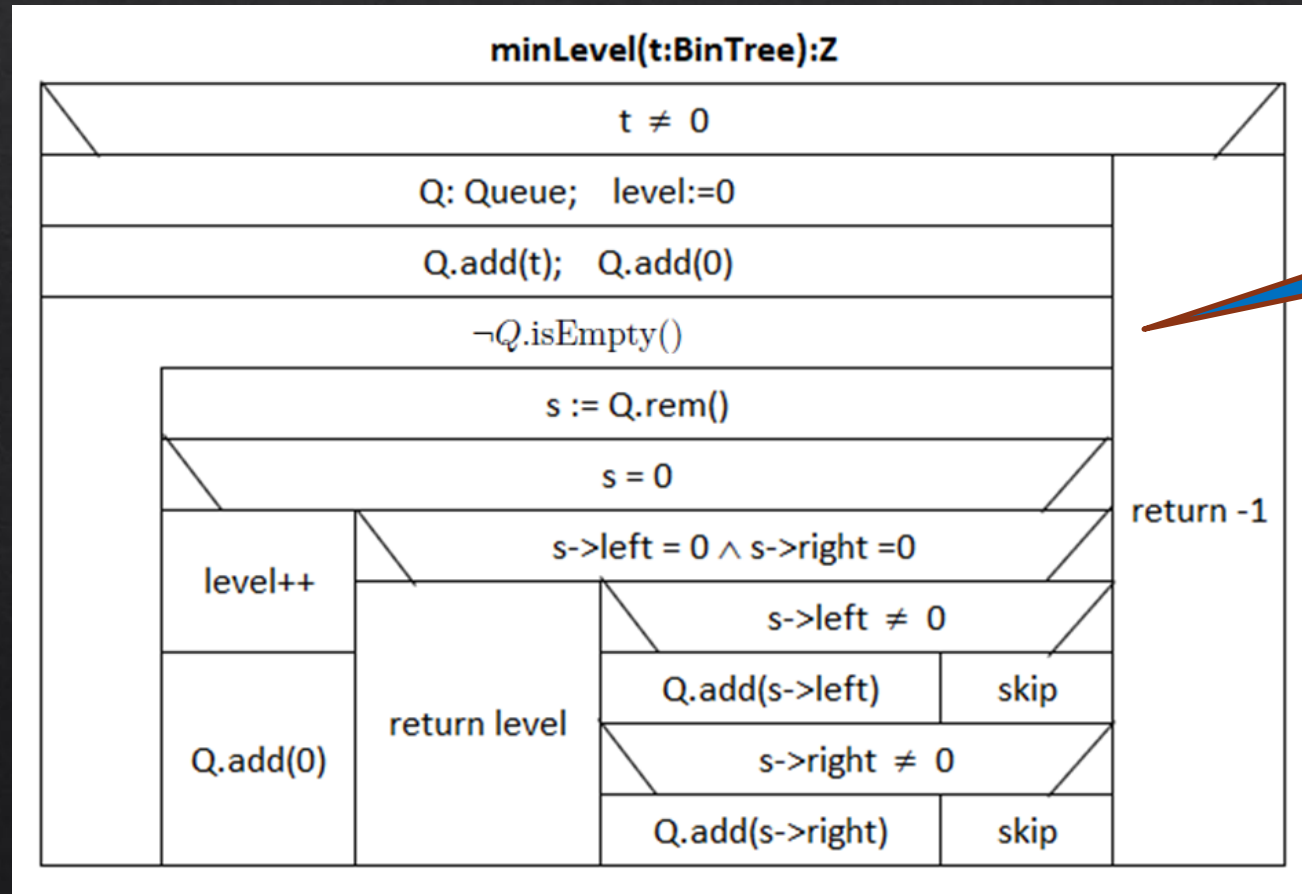
Solution with the first approach (with end mark)



Task:

Which is the closest level to the root
at which we can find a leaf?

Closest leaf



Why isn't it an infinite loop?

This is actually a selection problem: if the tree is not empty it always has a leaf, so the search will always reach a leaf. Since we are processing the elements level-wise, the first leaf we reach is going to give the answer.

Question:

- ◆ Modify the level order traversal algorithm: use a stack instead of a queue.
- ◆ What do we get?
- ◆ An iterative version of one of the previous traversals.
- ◆ Which one?

Preorder

